Q2.1

Two generators are considered equivalent if and only if they return the same output for every index n.

Q2.3

Both Fib1 and Fib2 are generators that produce the Fibonacci sequence.

Fib1 uses an iterative approach, based on the standard recursive definition of the Fibonacci numbers Fib2 uses Binet's formula, a closed-form expression that directly computes the Fibonacci number at position n, and casts the result to an integer.

Since Binet's formula is mathematically proven to yield the same Fibonacci numbers as the recursive method (when properly rounded), both generators return the exact same output for every index n. Therefore, Fib1 and Fib2 are equivalent generators.

Q3.2b

We will proof by induction on the length of lst1.
<u>"Base":</u> for lst1 of length 0, the value of (append lst1 lst2) is lst2 and the value of (append$ lst1 lst2 cont) is (cont lst2), which implies (append$ lst1 lst2 cont) = (cont (append lst1 lst2)).

<u>"Induction assumption":</u> We assume the proposition holds for lst1 of length n, and show the proposition holds for lst1 of a length n+1.

<u>"Induction Step":</u> According to the code, the value of (append lst1 lst2) is (cons (car lst1) (append (cdr lst1) lst2)), and the value of (append$ lst1 lst2 cont) is (append$ (cdr lst1) lst2 cont2), where cont2 is continuation procedure "(lambda (app-res) (cont (cons (car lst1) app-res))))".
Since the first operand of (append$ (cdr lst1) lst2 cont2) is a list of length n, according to induction assumption we get that (cont2 (append (cdr lst1) lst2)) = (append$ (cdr lst1) lst2 cont2).

So "(cont (cons (car lst1) (append (cdr lst1) lst2)))=(append$ (cdr lst1) lst2 cont2)"
and then we will get that "(cont (append lst1 lst2)) = (append$ (cdr lst1) lst2 cont2)",
and then we will get that "(cont (append lst1 lst2)) = (append$ lst1 lst2 cont)".

Q5.1

a. unify[t(s(s), G, s(U), p, t(K), s), t(s(G), G, K, p, t(K), U)]

A = t(s(s) , G , s(U), p, t(K), s)
B = t(s(G) , G , K , p, t(K), U)

S = { G = s }

AoS = t(s(s) , s , s(U), p, t(K), s)
BoS = t(s(s) , s , K , p, t(K), U)

S = { G = s , K = s(U) }

AoS = t(s(s) , s , s(U) , p, t(s(U)) , s)
BoS = t(s(s) , s , s(U) , p, t(s(U)) , U)

S = { G = s , K = s(s) , U = s }

AoS = t(s(s) , s , s(s) , p, t(s(s)) , s)
BoS = t(s(s) , s , s(s) , p, t(s(s)) , s)

Answer = { G = s , K = s(s) , U = s }

b. unify[p([[W | V] | [V | k]]), p([[v | V] | W])]

A = p([[W | V] | [V | k]])

B = p([[v | V] | W])

[W | V] = [v | V] => W = v => S = { W = v }

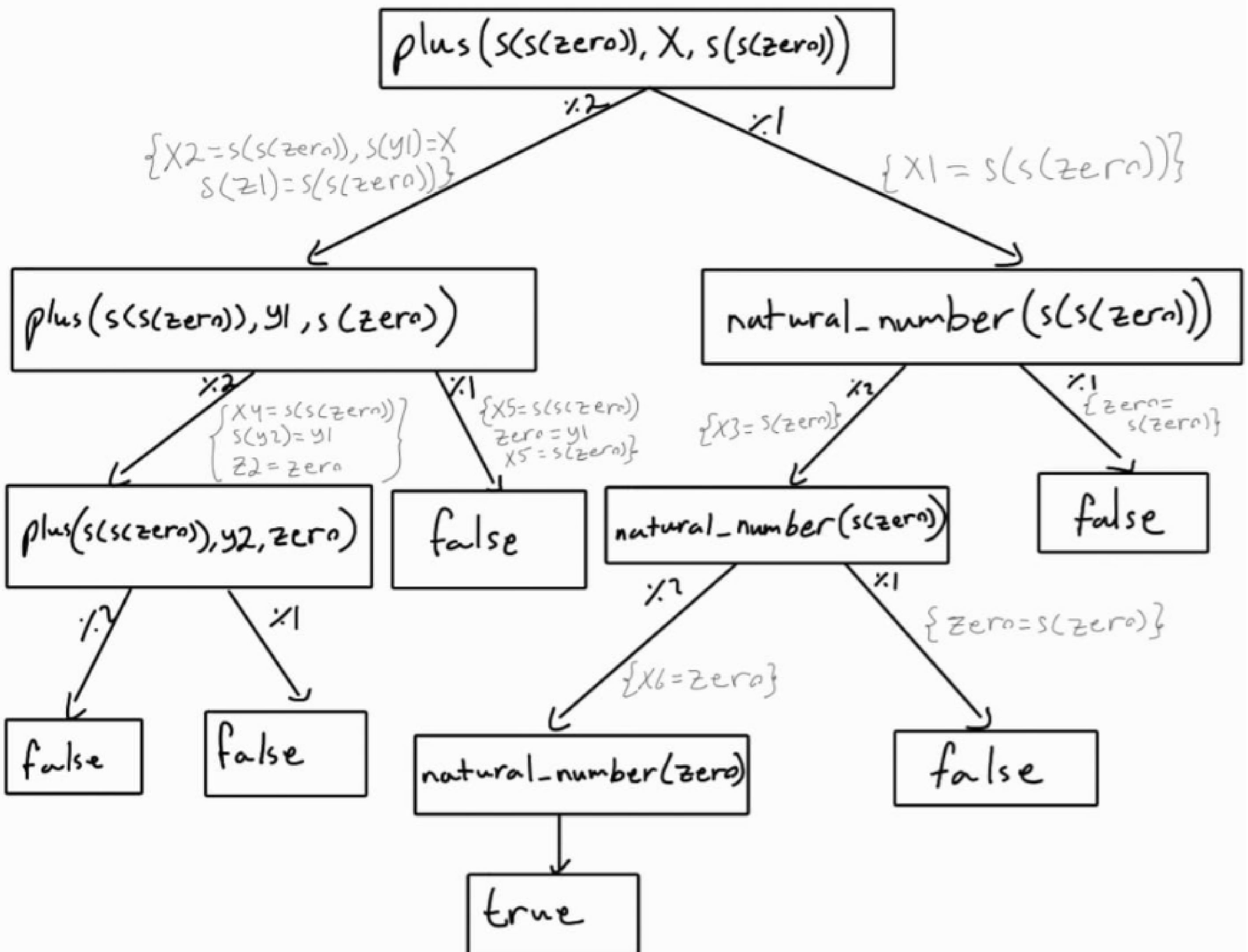
AoS = p([[v | V] | [V | k]])

BoS = p([[v | V] | v])

[V | k] = v

Answer: fails on list equals constant

Q5.3



*The tree is finite tree and success tree.